

PROJECT REPORT

Integrated College Management Backend System

Submitted By

Krishna Abhang

Student ID: 24BCE10678

Program: B.Tech Computer Science Engineering

Submission Date: June 2026

Table of Contents

1. Project Abstract	
2. Technology Stack & Prerequisites	
3. Database Architecture & Entity Mapping	
4. Implementation Details & API Routing	
5. System Testing & Output Screenshots	
6. Conclusion	

1. PROJECT ABSTRACT

The Integrated College Management System is a robust, server-side RESTful API engineered to centralize core administrative operations within an educational institution. Developed using Java, Spring Boot, and MongoDB, the system executes comprehensive CRUD (Create, Read, Update, Delete) operations across four primary entities: Departments, Staff, Students, and Library Books. The system deliberately operates as a decoupled backend layer, demonstrating advanced NoSQL relational mapping—including One-to-Many department allocations and Many-to-Many library borrowing tracking—to provide a scalable foundation for campus administration.

2. TECHNOLOGY STACK & PREREQUISITES

Category	Technology / Version	Purpose
Core Language	Java Development Kit (JDK) 21 LTS	Primary programming language
Framework	Spring Boot 3.2.3	REST API creation and embedded Tomcat server
Database	MongoDB 7.x (Community) & Spring Data	NoSQL data persistence and Object-Document Mapping
Build Tool	Apache Maven 3.9.x	Dependency management and application compilation
Utilities	Project Lombok	Boilerplate code reduction (@Data, @AllArgsConstructor)
Testing	Postman / Windows PowerShell	API endpoint execution and validation

3. DATABASE ARCHITECTURE & ENTITY MAPPING

The system utilizes MongoDB's flexible document model, employing ObjectIds as foreign key references to maintain relational integrity without traditional SQL joins.

- **Departments (Master Entity):** Stores core academic branch data. Generates the primary `_id` used by other collections.
- **Staff (One-to-Many):** Contains a `departmentId` field, logically linking multiple staff members to a single department.
- **Students (One-to-Many):** Contains a `departmentId` field, linking the student to their respective academic branch.
- **Books & Borrowing (Many-to-Many):** The Student entity maintains an array of strings called `borrowedBooks`. When a book is borrowed, its `ObjectId` is injected into this array, and the Book entity's `isAvailable` flag is toggled to false.

4. IMPLEMENTATION DETAILS & API ROUTING

The application follows a standard layered architecture (Models, Repositories, Services, and Controllers). The following endpoints were configured and exposed on `localhost:8081`:

Module	Endpoint	HTTP Method
Department	<code>/api/departments</code>	POST, GET, PUT, DELETE
Staff	<code>/api/staff</code>	POST, GET, PUT, DELETE
Student	<code>/api/students</code>	POST, GET, PUT, DELETE
Books	<code>/api/books</code>	POST, GET, PUT, DELETE
Business Logic	<code>/api/students/{id}/borrow/{bookId}</code>	POST

5. SYSTEM TESTING & OUTPUT SCREENSHOTS

The system was executed locally using the Maven wrapper (. \mvnw.cmd spring-boot:run). The following tests validate the system's data ingestion, relational mapping, and transactional integrity.

Test 1: Master Entity Creation (Departments)

Action: Executed a POST request to create a new academic department. This establishes the foundational ID for the database.

```
PS E:\My projects\Project MongoDB> $body = '{"deptName": "Computer Science", "deptCode": "CSE", "hodName": "Dr. Smith"}'
PS E:\My projects\Project MongoDB> Invoke-RestMethod -Uri "http://localhost:8081/api/departments" -Method POST -ContentType "application/json" -Body $body
```

id	deptName	deptCode	hodName
6a33de996be5f03a6e62d075	Computer Science	CSE	Dr. Smith

Test 2: One-to-Many Mapping Validation (Students)

Action: Registered a new student, passing the Department ID from Test 1 into the JSON payload to prove the entities are linked.

```
PS E:\My projects\Project MongoDB> $body = '{"studentName": "Krishna Abhang", "rollNumber": "24BCE10678", "departmentId": "PASTE_DEPT_ID_HERE", "borrowedBooks": []}'
PS E:\My projects\Project MongoDB> Invoke-RestMethod -Uri "http://localhost:8081/api/students" -Method POST -ContentType "application/json" -Body $body
```

id	: 6a33deb56be5f03a6e62d076
studentName	: Krishna Abhang
rollNumber	: 24BCE10678
departmentId	: PASTE_DEPT_ID_HERE
borrowedBooks	: {}

Test 3: Populating Library Inventory (Books)

Action: Added a new textbook to the database. The system automatically sets the availability flag to true.

```
PS E:\My projects\Project MongoDB> $body = '{"title": "Operating System Concepts", "author": "Galvin", "isbn": "978-111", "isAvailable": true}'
PS E:\My projects\Project MongoDB> Invoke-RestMethod -Uri "http://localhost:8081/api/books" -Method POST -ContentType "application/json" -Body $body

id      : 6a33ded66be5f03a6e62d077
title   : Operating System Concepts
author  : Galvin
isbn    : 978-111
available : True
```

Test 4: Transactional Business Logic (Borrowing a Book)

Action: Triggered the custom borrowing endpoint. The Java Service layer successfully appended the Book ID into the Student's array and marked the book as unavailable.

```
PS E:\My projects\Project MongoDB> Invoke-RestMethod -Uri "http://localhost:8081/api/students/6a33deb56be5f03a6e62d076/borrow/6a33ded66be5f03a6e62d077" -Method POST

id          : 6a33deb56be5f03a6e62d076
studentName : Krishna Abhang
rollNumber  : 24BCE10678
departmentId : PASTE_DEPT_ID_HERE
borrowedBooks : {6a33ded66be5f03a6e62d077}
```

Test 5: Full Data Persistence Verification

Action: Executed a GET request to fetch all students, proving that the multi-collection updates were permanently saved to MongoDB.

```
PS E:\My projects\Project MongoDB> Invoke-RestMethod -Uri "http://localhost:8081/api/students" -Method GET

id          : 6a33deb56be5f03a6e62d076
studentName : Krishna Abhang
rollNumber  : 24BCE10678
departmentId : PASTE_DEPT_ID_HERE
borrowedBooks : {6a33ded66be5f03a6e62d077}
```

6. CONCLUSION

The successful development and testing of the Integrated College Management Backend System demonstrates a highly efficient approach to institutional data architecture. By utilizing Spring Boot and MongoDB, the system ensures rapid data processing while maintaining strict relational integrity without the rigid constraints of a traditional SQL database. The successful execution of the Library Borrowing module specifically highlights the API's capability to enforce complex, multi-document business rules autonomously. This backend serves as a secure, production-ready foundation that can seamlessly integrate with any modern web, mobile, or desktop frontend application.